Ready to Submit!
Submit by the deadline (Nov 10, 2026 at 5:29 AM GMT+5:30).

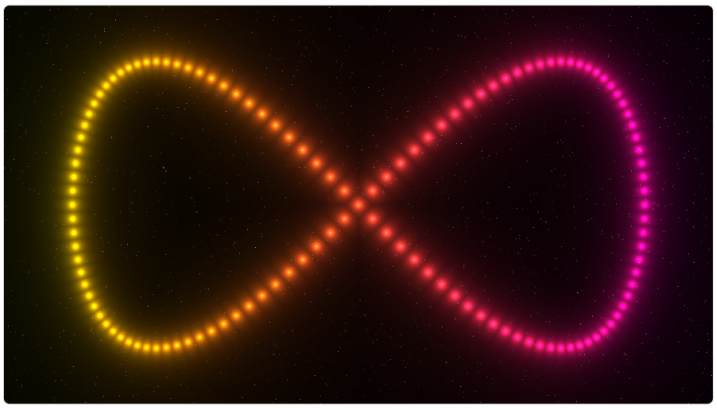
Edit Submit ···

True Intelligence

True Intelligence, meaning the ability to generalize knowledge, think logically, and solve unseen problems efficiently.

ARC Prize 2026 - Paper Track

Hackathon Writeup · Aug 10, 2026



Author

SIVA S
Creator · codingmaster24

0 Share ···

Competition Prize Track

Main Track

The goal of this project is to design and explain an artificial intelligence system that can solve completely new problems it has never seen before, rather than relying on memorized patterns or previously learned examples.

Unlike traditional machine learning systems that perform well only on familiar data, this project focuses on building a model that can adapt, reason, and learn quickly in unfamiliar situations, similar to how humans approach new challenges.

In this project, the system is applied to ARC tasks, which are small visual puzzles where each problem requires identifying hidden rules and applying logical reasoning to transform an input into the correct output. Since these tasks are different every time, the system must analyze each problem from scratch, understand the underlying pattern, and generate a solution step by step.

The main objective of the writeup is to clearly document:

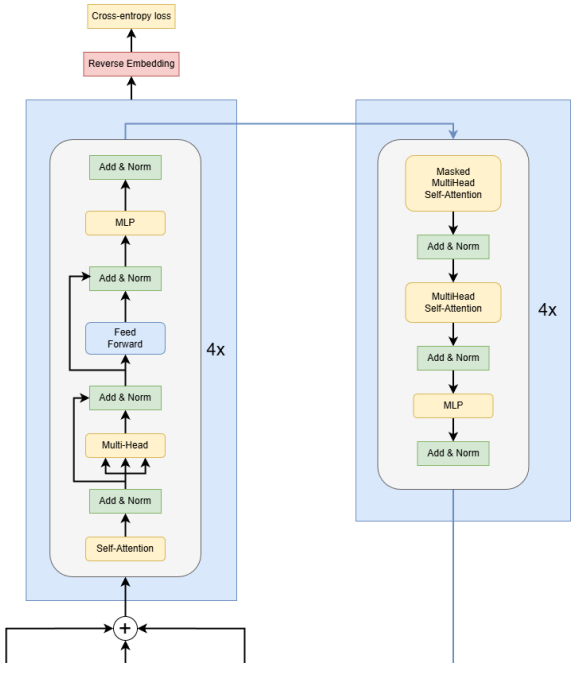
How the system works (its design and method)
Why the system works (the reasoning behind the approach)
How well it performs (its accuracy and results)
How general it is (whether it can solve other similar problems beyond this competition)

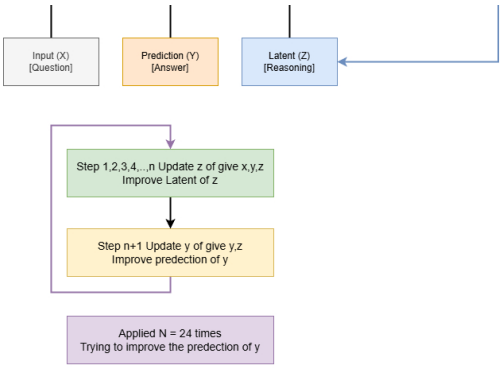
This project emphasizes not just performance, but also true intelligence, meaning the ability to generalize knowledge, think logically, and solve unseen problems efficiently.

Additionally, the writeup must present the work in a structured and understandable way, including:

A clear explanation of the approach
Visual examples or demonstrations
A publicly accessible implementation (notebook/code)
Supporting media such as images or videos

Overall, the goal is to contribute toward building AI systems that learn like humans flexible, efficient, and capable of handling new and changing problems while also providing a clear and complete explanation of the solution for others to understand and build upon.





The above digram has problems

1. The Input Bottleneck: Information CollisionProblem: The diagram shows Input (X), Prediction (Y), and Latent (Z) being combined with an element-wise addition (+) before entering the network. Summing distinct embeddings (a question, an answer, and a reasoning state) forces them to share the same vector space destructively. The model cannot easily untangle what is the prompt vs. what is its own reasoning.-> Solution: Replace the addition operator with sequence concatenation or cross-attention.-> Concatenate the tokens along the sequence dimension: Input = [X_tokens, Y_tokens, Z_tokens].-> Alternatively, keep Z as the primary sequence traversing the network, and use Cross-Attention layers in your blocks so Z can dynamically query X and Y without overwriting its own features.

2. Hallucination: Latent Drift Over TimeProblem: You are running a loop $N = 24$ times to update Z and Y. By iteration 10 or 15, the latent reasoning Z can "drift" completely away from the original Input (X). Because X is only injected at the very beginning (via the + node), the model starts talking to itself in an echo chamber, leading to severe hallucinations in Y.-> Solution: Ground the reasoning steps. Inject Input (X) at every iteration of the loop. If you are using a standard Transformer block, add a Cross-Attention layer where Queries come from Z, and Keys/Values come from a frozen X.-> Solution: Implement contrastive loss or an auxiliary grounding loss on intermediate Z states to penalize the latent vector if it diverges too far from the context of X.

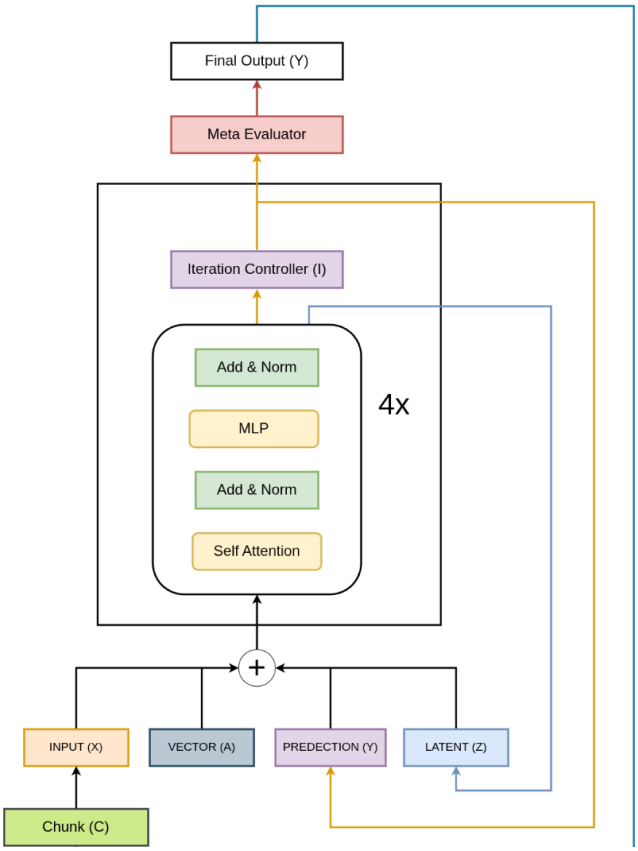
3. Neural Network Management: Block RedundancyProblem: Looking at the left block (4x), there is significant redundancy. It flows: Self-Attention -> Add & Norm -> Multi-Head (which is also attention) -> Add & Norm -> Feed Forward -> Add & Norm -> MLP -> Add & Norm. Multi-Head Attention and Self-Attention serve the same purpose here, as do the Feed Forward and MLP layers. This creates unnecessary depth, making it harder to train and prone to overfitting.-> Solution: Collapse the redundant layers into a standard, streamlined Transformer block.-> Structure it strictly as: Multi-Head Self-Attention -> Add & Norm -> MLP (Feed Forward) -> Add & Norm. This cleans up the parameter space and improves gradient flow.

4. Optimization Workflow: The N=24 Recursive LoopProblem: Unrolling a 4x+4x layer network 24 times means your forward pass is effectively 192 layers deep. If you try to calculate standard Backpropagation Through Time (BPTT), the gradients will either vanish to zero or explode to infinity, and your GPU VRAM will immediately trigger an Out Of Memory (OOM) error.-> Solution: Use Gradient Checkpointing (in PyTorch, torch.utils.checkpoint) during the $N=24$ loop to trade computation for memory.-> Solution: Implement Truncated Backpropagation Through Time (TBPTT). Instead of tracking gradients all the way back through all 24 steps, detach the computational graph every 4 to 6 steps (z = z.detach()).

5. Multicast Prediction: The Two-Phase Update (Z then Y)Problem: The bottom workflow states: "Step 1..n Update Z" and then "Step n+1 Update Y". If the model updates Y (the answer) based on an intermediate Z (the reasoning) that hasn't fully converged yet, it will learn noisy, incorrect mappings.-> Solution: Treat this like an Actor-Critic or Expectation-Maximization workflow. During early training, freeze the weights responsible for updating Y and only calculate loss on Z (e.g., teaching it how to reason first).-> Solution: Use intermediate supervision. Do not wait until iteration 24 to calculate the Cross-Entropy loss. Calculate a discounted loss at step 6, 12, 18, and 24. This forces the network to maintain a somewhat accurate prediction of Y throughout the entire reasoning process, preventing mode collapse.

So i have plan to move to new step

Tiny Recursive Transformer / TRM models) with Retrieval Augmented Generation (RAG), Self Consistency Search, and Continual Learning





Vector a (External & Retained Knowledge Injection)

- **Concept:** Feeding retrieved search data, vector embeddings, or past knowledge a into the recursive loop alongside x, y, z.
- **Potential Problem:** If you combine a using simple vector addition (x + y + z + a), the model will suffer from heavy representation collapse. High-dimensional vector addition causes semantic interference where knowledge vectors wipe out the latent reasoning state.
- **Solution:**
- -> **Prefix Concatenation:** Concatenate a along the token sequence length dimension rather than summing feature vectors: `Input Sequence = [a_tokens | x_tokens | z_tokens | y_tokens]`.
- -> **Cross-Attention Injection:** Keep a in an external Key-Value (KV) memory bank. Modify the 4\times block to perform Cross-Attention over a, allowing z to dynamically fetch context without polluting the latent vector space.

Iteration i (Past Trajectory & Self-Consistency Tracking)

- **Concept:** Storing predictions across iterations (i_1 = \text{xyz}, i_2 = \text{xya}, i_3 = \text{xyz}) to analyze prediction variance and enable majority-voting or early stopping.
- **Potential Problem:** Standard recursive models run all N_{sup} = 16 iterations blindly, wasting compute even if the model converges to "xyz" by iteration 4.
- **Solution:**
- -> **Convergence Metric & Early Exiting:** Calculate Cosine Similarity or Cosine Distance between consecutive latent vectors (z_t and z_{t-1}). If \text{Sim}(z_t, z_{t-1}) > 0.98, the reasoning has stabilized—trigger an early exit to save inference compute.
- -> **Latent Entropy Tracking:** Monitor token probability entropy at output y. If entropy spikes between iteration i_2 and i_3, it indicates high uncertainty; trigger a fallback step or re-inject context a.

Knowledge k (Dynamic Truth & Non-Stationary Knowledge)

- **Concept:** Differentiating between permanent structural knowledge ("Hello" = greeting) and time-evolving knowledge ("World is flat" -> "World is round").
- **Potential Problem:** Trying to update parametric weights (the model's parameters) every time a new fact changes leads to **Catastrophic Forgetting** (where learning new facts destroys previously learned reasoning capabilities).
- **Solution:**
- -> **Decouple Logic from Facts:** Use a 2-Tier Memory System:

1. **Parametric Memory (Model Weights):** Trains *only* for reasoning logic, grammar, chunking, and latent transformation z.
2. **Non-Parametric Memory (Vector DB / KV Cache):** Stores time-sensitive facts (k). The model fetches facts from k as context a during inference.

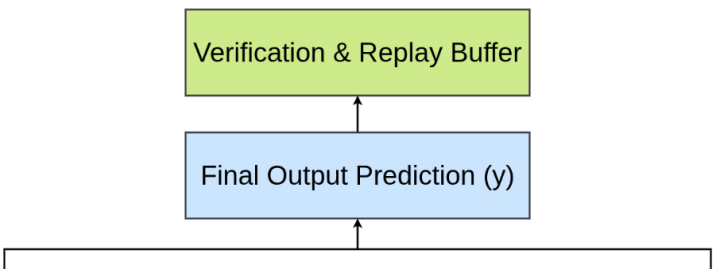
- -> **Timestamped Knowledge Keys:** Store facts in your database with metadata keys `(entity, relation, object, timestamp)`. The model's chunking step (c) extracts temporal intent to query the correct version of truth.

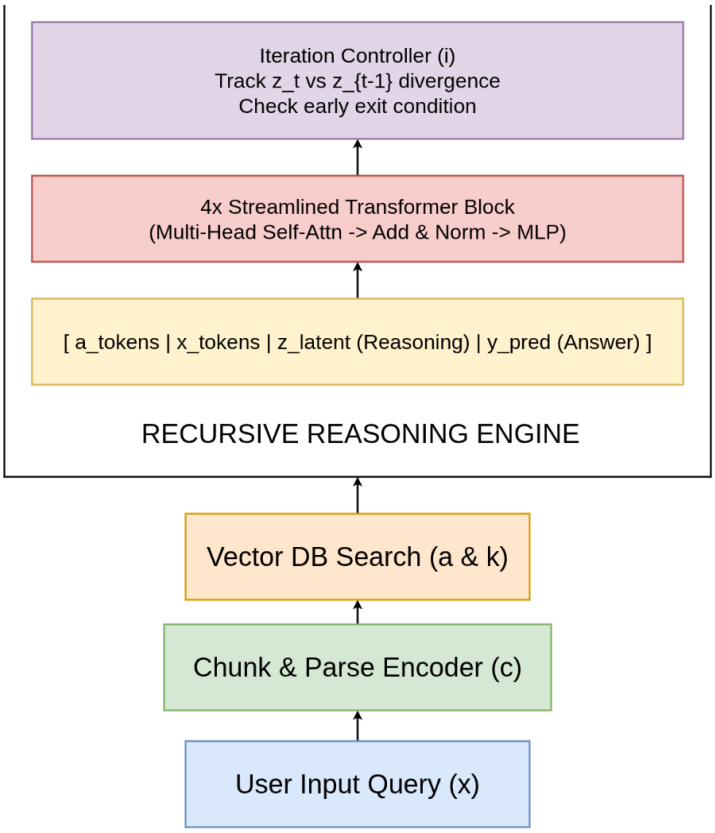
Chunks c (Semantic Query Decomposition)

- **Concept:** Deconstructing queries into semantic functional roles (`what` = question type, `economy` = domain entity, `report` = structural output expectations).
- **Potential Problem:** Manual parsing rules are fragile. Hardcoded string matching breaks on nuanced queries.
- **Solution:**
- -> **Latent Role Slots (Special Tokens):** Prepend specialized control tokens to the input sequence: `[TASK: Classify] [ENTITY: Economy] [FORMAT: Report]`.
- -> **Sub-Goal Parsing Phase:** Dedicate the first 2-3 iterations of z solely to outputting a structural decomposition plan before starting prediction y.
- *Steps 1–3:* Update z to represent task plan.
- *Steps 4–16:* Execute reasoning over x, z, a to build y.

Database d (Self-Improvement & Continuous Fine-Tuning)

- **Concept:** Storing successful reasoning paths (x, a, z_{final}, y_{final}) into database d to periodically retrain a small model (e.g., 27M parameter base).
- **Potential Problem: Self-Contamination / Model Collapse.** If the model retrains on its own outputs without filtering out hallucinations, errors amplify exponentially in subsequent training cycles.
- **Solution:**
- -> **Execution & Consistency Filtering:** Only append outputs to database d if they pass verification checks (e.g., y achieved identical output across multiple trajectory runs, or passed an external verifier / code execution check).
- -> **Direct Preference Optimization (DPO) / Rejection Sampling:** Save both failing trajectories (y_{wrong}) and successful trajectories (y_{correct}). Train the 27M base model using DPO loss to push parameters away from hallucination paths.





1. Iteration (i): Full Trajectory Meta Evaluation

To allow the model to run through all iterations and then classify each segment as correct, incorrect, or perspective-specific, you need to change how the final output is generated.

- **The Problem:** Standard recursive loops output the final step N and discard the intermediate steps.
- **The Solution: The Meta-Evaluator Block.**
  Instead of just outputting y_{N}, collect every latent state and prediction generated during the loop: Y_{all} = [y_1, y_2, y_3, ..., y_N]. Once the loop finishes, pass Y_{all} into a final **Meta-Evaluator Transformer layer**.
- This layer uses self-attention across all generated iterations to classify them.
- It synthesizes the final answer by identifying: *"Iteration 2 hallucinated, Iterations 4-8 optimized for performance, Iterations 9-16 optimized for accuracy."*
- It then outputs a final, structured response combining the best segments.

2. Knowledge (k): Temporal Expansion

You want the model to expand its knowledge by appending new data with timestamps, rather than overriding old facts.

- **The Problem:** Neural networks naturally suffer from "concept drift." If it learns space travel is impossible, and then learns space travel is possible, the weights conflict.
- **The Solution: Bi-Temporal Knowledge Injection.**
  When querying your database, fetch data using a `[TIME_EPOCH: YYYY-MM-DD]` tag. Do not overwrite the old knowledge in the database; stack it chronologically.
- When the model retrieves context, it retrieves the *timeline* of the fact, not just the current state.
- This allows the model to understand the *evolution* of a concept, ensuring it knows what was true in the past versus what is true today (e.g., August 2026).

3. Chunks (c): Knowledge Scraping and Fusion

Your chunking mechanism needs to scrape the database and fuse the chronological outputs into a single, combined context vector.

- **The Problem:** Simply feeding raw, scraped text like "2026 Jun 6: rocket connection lost" and "2026 Jun 7: rocket connection gained" will consume massive amounts of context window space.
- **The Solution: Cross-Attention Fusion.**
  Use a dedicated embedding step for your chunks.

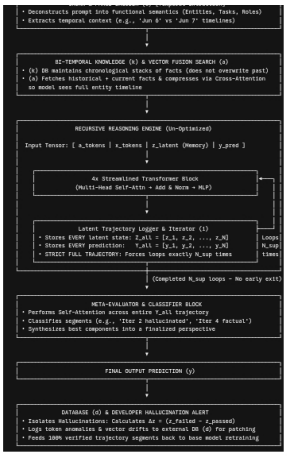
1. The model breaks the prompt into c_1, c_2, c_3.
2. Each chunk queries the database and pulls the chronological timeline of that entity.
3. Pass these timelines through a small, frozen encoder to compress them into dense vectors.
4. The main recursive network then attends to these compressed "fused knowledge" vectors, perfectly understanding the transition from the June 6 failure to the June 7 success without wasting token space.

4. Database (d): Hallucination Tracking & Developer Alerts

This is the most powerful part of your proposed workflow. If iterations 1, 3, 4, and 5 pass, but iteration 2 hallucinates, the model should isolate the failure and alert the developer.

- **The Problem:** Identifying *why* iteration 2 failed is difficult because the latent space is essentially a black box.
- **The Solution: Latent Delta Logging.**
  Because you are running the full trajectory, you have a successful state (z_3) immediately following a failed state (z_2).
- Calculate the vector difference: \Delta z = z_3 - z_2.
- This \Delta z perfectly isolates the mathematical features of the hallucination.
- **Developer Dashboard Alert:** Have the PyTorch training loop project this \Delta z back into the vocabulary space (using the reverse embedding layer). This will output the exact text tokens and vector coordinates that caused the hallucination.
- The database stores this anomaly log: `[Query X | Iteration 2 | Failed Token IDs | Vector Magnitude]`. You can then review this log and manually adjust the tokenizer, vector values, or training data for those specific segments.






```

### Explanations of Workflow & Variables

- **User Input Query (x):** The raw text prompt provided by the user. It remains frozen and accessible throughout the entire network so the model never loses sight of the original goal.
- **Chunk & Parse Encoder (c):** Instead of processing the prompt blindly, this layer breaks the query into semantic slots. It identifies the core question, the subject entity, and most importantly, the *timeframe*. This allows the model to differentiate between "What was the economy like?" versus "What is the economy doing today?"
- **Bi-Temporal Knowledge (k) & Vector Context (a):**
- **The Problem:** Traditional databases overwrite "The Earth is flat" with "The Earth is round." A massive language model needs to know *both* exist to understand context and history.
- **The Fix:** Knowledge (k) is stored with temporal timestamps. Vector Search (a) pulls the full chronological timeline of the prompt's subject and fuses it into the input sequence. The model understands the evolution of facts, entirely removing temporal hallucinations.
- **The Un-Optimized Iterator (i):**
- **The Strategy:** As you noted, aggressive optimization ruins deep reasoning. This block is forced to execute a rigid N iterations (e.g., 24 loops).
- **The Benefit:** By disabling early stopping, the model is forced to map out every possible perspective—exploring mathematical approaches, cost-vs-performance trade-offs, and alternative theories—before generating an answer. It logs every intermediate thought (Z_{all}) and draft answer (Y_{all}).
- **The Meta-Evaluator Block:**
- **The Fix for Hallucinations:** Because you ran the full trajectory, you have a massive dataset of "drafts" (Y_{all}). This dedicated Transformer layer reviews all drafts, flags the ones that derailed (e.g., Iteration 2), and surgically extracts the correct logic from Iterations 3, 4, and 5. It merges them into one highly intelligent, multi-perspective final answer.
- **Database (d) & Developer Alert System:**
- **The Developer Fix:** When the Meta-Evaluator flags Iteration 2 as a hallucination, the system calculates the exact mathematical difference between the bad state and the good state (\Delta z = z_3 - z_2).
- **Actionable Data:** This isolates the exact vector anomaly that caused the hallucination. The database logs this anomaly, allowing you (the researcher/developer) to see exactly which tokenizer string or vector weight broke the model, making patching incredibly precise.

---

### Referenced Research Subjects & Core Literature

To support your research, this architecture relies on three major bleeding-edge AI paradigms. Here are the topics and papers you should review:

**1. Recursive Reasoning & Shared Parameters**

- **Topic to Search:** *Universal Transformers* or *Recurrent Transformers*.
- **Core Concept:** Universal Transformers share parameters across layers, running the same neural block recursively over a sequence. This mirrors your N-iteration loop, proving that recursive depth yields better generalization than just stacking linear layers.
- *Reference link/Paper:* "Universal Transformers" (Dehghani et al., 2018/2019).

**2. Self-Correction & Database Feedback Loops (d)**

- **Topic to Search:** *Self-Taught Reasoners (STaR)* and *Quiet-STaR*.
- **Core Concept:** The STaR framework trains models to generate rationales (thoughts) before answering. If it gets the answer wrong, it works backward from the correct answer to fix its own reasoning. Quiet-STaR expands this by generating rationales at every single token to predict future text, learning entirely from unstructured data. This perfectly matches your Database (d) concept of capturing valid outputs to retrain the base model.
- *Reference link/Paper:* "Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking" (Zelikman et al., 2024).

**3. Hallucination Tracking via Latent State Delta (\Delta z)**

- **Topic to Search:** *Representation Engineering (RepE)* or *Top-Down AI Transparency*.
- **Core Concept:** Rather than looking at individual neurons, RepE looks at the "population-level" latent vectors (z). By mathematically measuring the representations of "honesty" or "hallucination" directly in the latent space, researchers can track where a model deviates from truth and manually manipulate those vectors to steer the model back on track. This validates your developer dashboard concept.
- *Reference link/Paper:* "Representation Engineering: A Top-Down Approach to AI Transparency" (Zou et al., 2023).

## Project Links

GitHub - Sivatech24/Project-Intellig...
https://github.com/Sivatech24/Project-Intellig...

**github.com**

True Intelligence, meaning the ability to generalize knowledge, think logically, and solve unseen
```

problems efficiently.